

Institute of Information Science, Academia Sinica

2025 Summer Internship Report (2025/07/01~2025/08/29)

Name : Yu-Yan Zheng 鄭宇彥	School : National Taiwan University	Major : Electrical Engineering
Lab Host : Li Su 蘇黎	Email: maxzheng0302@gmail.com	
Brief description of internship content	The internship involves paper survey, code implementation and problem solving. Topics ranging from music generation, inpainting, and segmentation tasks. The main machine learning techniques involve latent diffusion, attention mechanism, and auto-encoders. Furthermore, there is also signal processing on music information.	
Was it what you expected? Why or why not?	The whole internship experience is beyond my expectation. With weekly meetings and report, I learn a lot from professor's feedbacks and my colleague's representations. In addition, the problem-solving process is complex yet very interesting. I'm grateful that I have the valuable chance to enhance myself.	
Overall self-evaluation	The enhancement on paper survey skill: During this summer internship on IIS, I nearly read up to 40 papers. I'm able to extract useful information and spot the core contribution each paper provides. It really helps the time when I start writing codes or try to solve some problems. I can in turn use any method I've learned from those papers. The advancement of coding: With lots of time spent on code analysis, I'm more familiar with Python writing, especially on pytorch implementation, numpy operations, matplotlib techniques, and how to introduce GPU execution into our code as well. In addition, since I have to work on lab's server, I learn how to use docker to manage my own environment. However, I also realize it's very difficult and time consuming to complete a project and even to publish a paper. It's not about some attempts or analysis, but huge amounts of experiments, deep and broad evaluations, and the most importantly, the novelty in our work, which can truly make contributions to the academic community.	
Score for myself	100/100	

【Essay】 1000-1500 words

I categorize my internship into three major sections, which are listed below.

I. Paper Survey

- Introduction

Machine learning methods on music generation are developing rapidly given the fact that music and image are somewhat of the same nature, while we can't deny that deep learning on computer vision is attaining perfection. Among a great variety of DL techniques, **diffusion model** inherits the essence of variational auto-encoders, integrates the attention mechanism, and exploits a special diffusion/denoising process, whose mathematical expressions, stochastic differential equations, can be theoretically connected with score based generative model. Diffusion model is capable of not only performing high quality generation tasks but also solving manifold inverse problems. This versatile, scalable, and flexible tool is used on music generation as a matter of course.

There are also two categories for music representations. Symbolic features focus on representing the music

in tokens or events, like **MIDI files**. This type of music representation can be converted to **piano rolls** or music sheets easily and also more interpretable for human or some on-shelf libraries. In addition, symbolic music data suffers less information loss during format transformation as long as time unit is correctly specified. On the other hand, we can also process music data directly in **waveform**, the signal array revealing the amplitude sequence versus each time step. Conventionally, short time Fourier Transform is a typical subsequent processing method to convert waveform into **spectrogram**, or into mel-spectrogram if the frequency axis is sampled in logarithm according to 12 equal temperament principle, which corresponds to the sensitivity of human's auditory perception. In contrast to symbolic music representation, the spectrogram or waveform is more applicable and instantly accessible from any form of music, ranging from live concert, instrument recording, or any .mp3 or .wav files. However, the generated magnitude spectrogram won't have phase information, therefore a vocoder is required to reconstruct waveform with promising quality.

The strengths and weaknesses of two aforementioned methods complement each other, and interestingly, it's possible to represent music in **image-like data structure** such as piano roll or spectrogram respectively.

- **Music Generation**

There are various different ML architectures based on diffusion model work on music generation given that diffusion model performs so good on high resolution image generation. For example, ACE-step aims to generate a whole song. Thus, it includes different encoders such as text, lyric, and speaker encoders to generate condition embeddings which is going to be cross-attended into latent diffusion model. In addition to flow matching loss, semantic alignment loss is also considered based on two encoders MERT and mHuBERT. For some recent works in 2025, some attempts involve integrating Mamba and wavelet transform blocks into UNet backbone, and doing instrument editing during a specific timestep during denoising process, which is guided by a lightweight classifier distilled from MERT.

Whole-song hierarchical generation of symbolic music is a paper with open-source codebase and perform nicely on well-structured music generation. It proposes hierarchical music language with four levels, from phrase, lead sheet to accompaniment. Each stage of diffusion model concentrates on a certain interpretable aspect of music composition, and high-level language is realized by stylistic specifications at lower levels since low-level music serves as the background conditions during training and generation. Furthermore, it's capable of producing long songs since autoregressive conditions are chosen from generated segments.

- **Music Segmentation**

Music segmentation belongs to **MIR** (Music Information Retrieval) tasks. The simplest task is to detect the boundaries within a song. The difficulty will rise when we'd like to classify each segment, in other words, to learn the relationship instead of only spotting the difference. A more advanced task falls on learning the hierarchical segmentation, which will occur in some professionally labelled music following rigorous rules. There comes another problem that it's relatively difficult to do supervised learning since segment-labelled music data is so scarce. SALAMI's label files are publicly accessible however the corresponding song files aren't provided due to copyright issues.

If we turn to **unsupervised music segmentation learning**, there are two general methods:

Unsupervised learning of deep features for music segmentation trains a convolutional neural network to transform audio representation into domain that is representative of musical structure. The optimization

objective is to minimize the triplet loss calculated per mini batch consisting of anchor, positive and negative examples. Ideally, positive examples symbol feature representations from the same music segment as the anchor, while negative examples will represent audio features from distinct music segments. The selection of those examples is mainly based on the distance between each segment. (Unfortunately, it's already an inaccurate hypothesis) After the meaningful embeddings (outputs of CNN) are extracted, they are used to construct SSM (self-similarity matrix) to detect boundaries.

Unsupervised symbolic music segmentation using ensemble temporal prediction errors provides another aspect on music segmentation. The backbone RNN model will try to predict the next symbolic representation based on previous input symbols, and then the peak detection algorithm will find the segment boundaries according to the prediction loss (cross entropy loss).

One of the great advantages of unsupervised learning is that it **doesn't require labelled dataset**, therefore can get training set easier, while the answer to segment boundaries is not quite obvious, sometimes even needing approximations or hypothesis which may over simplify or misinterpret the problem itself.

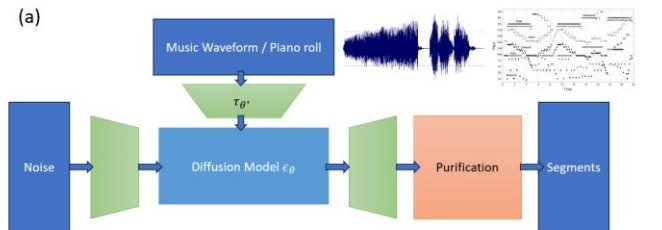
II. Implementation

After paper surveying, I initially come up with two different methods:

Future Works

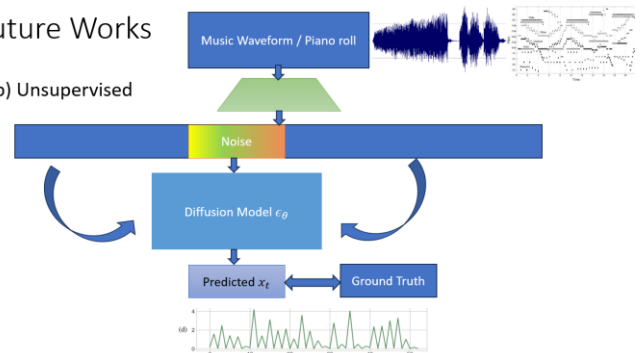
- Keep working on segmentation
- Some proposed model architecture:

(a)



Future Works

- (b) Unsupervised



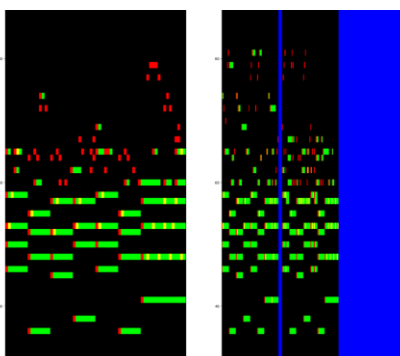
Consider the flexibility and scalability of unsupervised learning, my later work resembles proposal (b) more. My target is to find segment boundaries within a song, i.e. the input midi file.

- Used datasets:

1. **pop909's** aligned midi files from the repo of whole song hierarchical generation.
2. **pop1k7's** midi files which can be downloaded online.

- **Data process:**

Read the midi files and first convert to note matrix, consisting of elements: [start tick, note pitch, end tick]. Further scaling is to ensure that the time unit (tick) satisfies: 2 ticks/beat. The note matrix can be easily transformed into piano roll data. For each song, its piano roll representation has shape (C=2, W=length (tick), H=128). Channel number corresponds to **onset / sustain channels**, while length is literally the total number of ticks of the whole song. Height of 128 means there are **128 different pitches**.

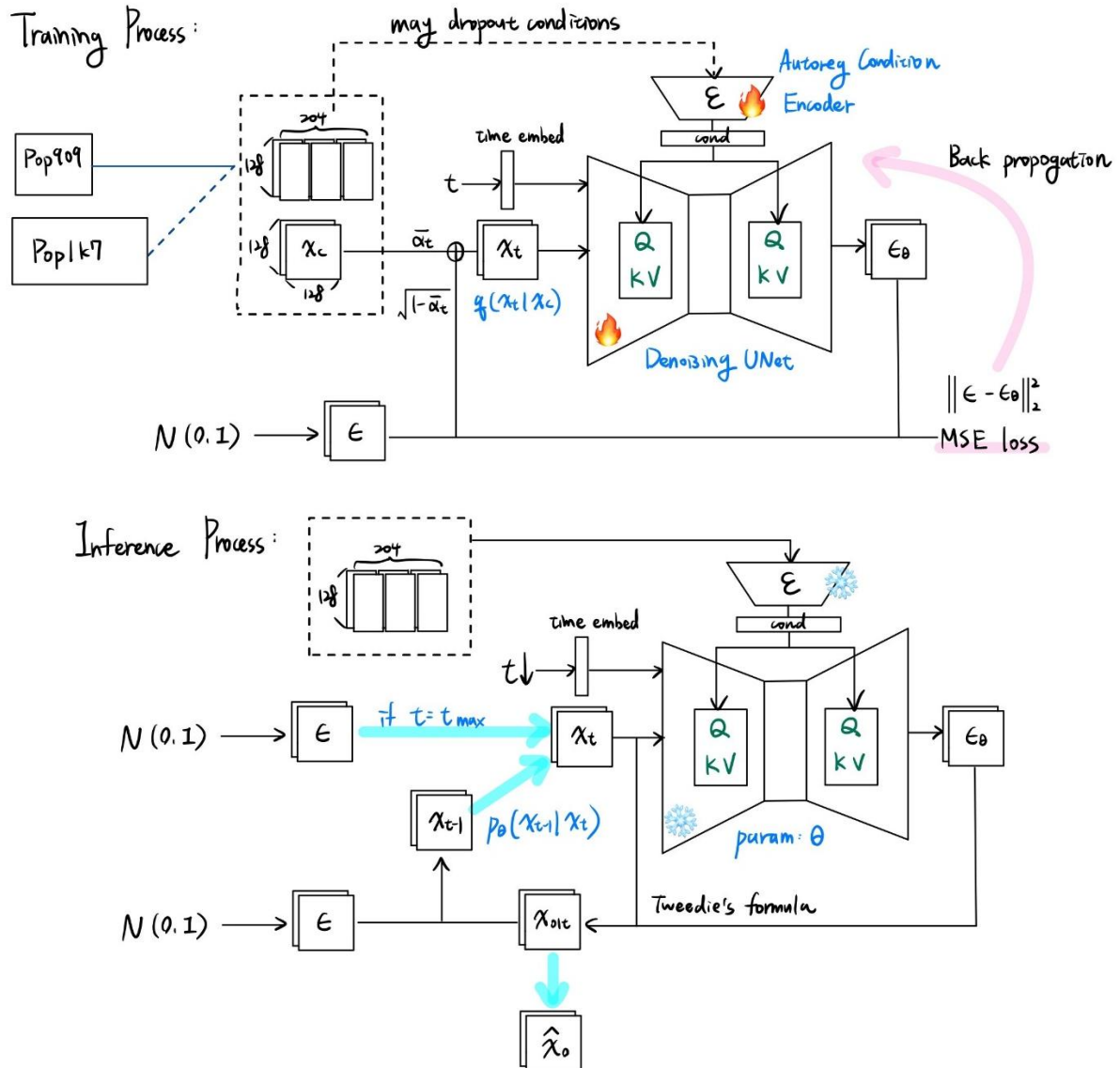


The sub figure on the left shows the piano roll of one music segment. red = onset channel, green = sustain channel. (yellow is overlap)

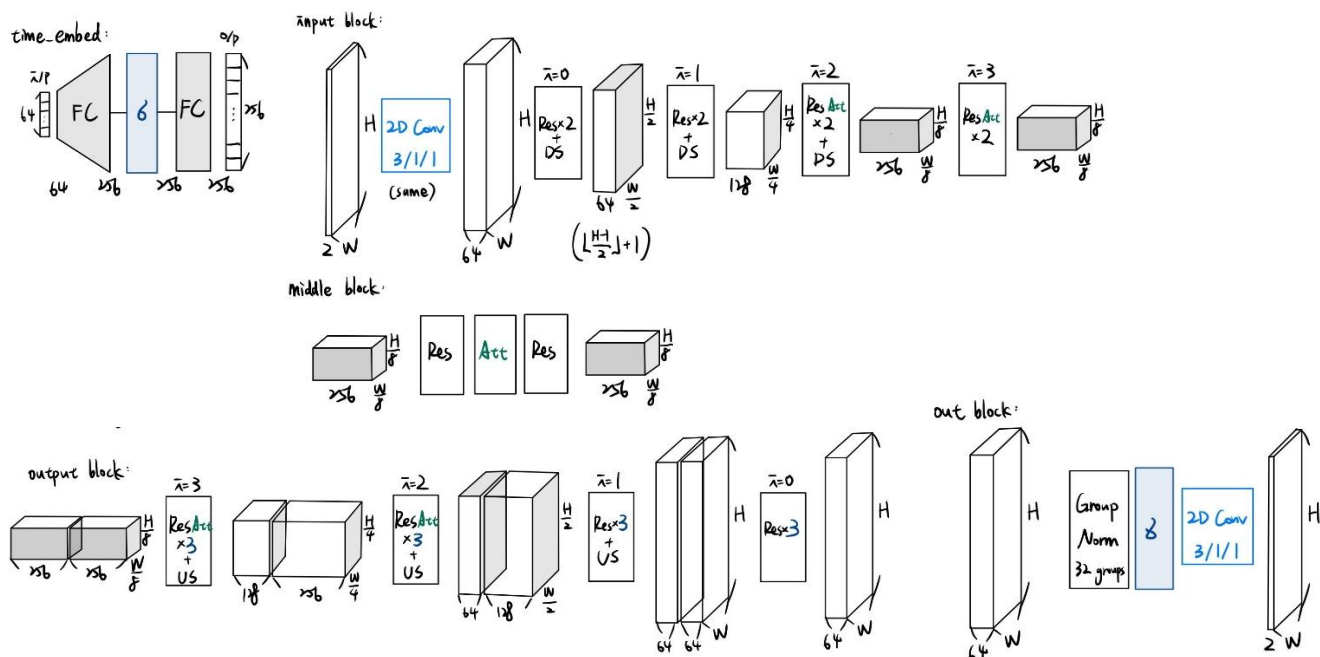
The sub figure on the right indicates the auto regressive conditions selected according to the current sequence. Auto regressive conditions are 0~3 (randomly) segments of length 64 concatenated together with -1 paddings (blue regions) before the current sequence within the same song. Starting positions can be the nearest or totally random.

- Model Architecture

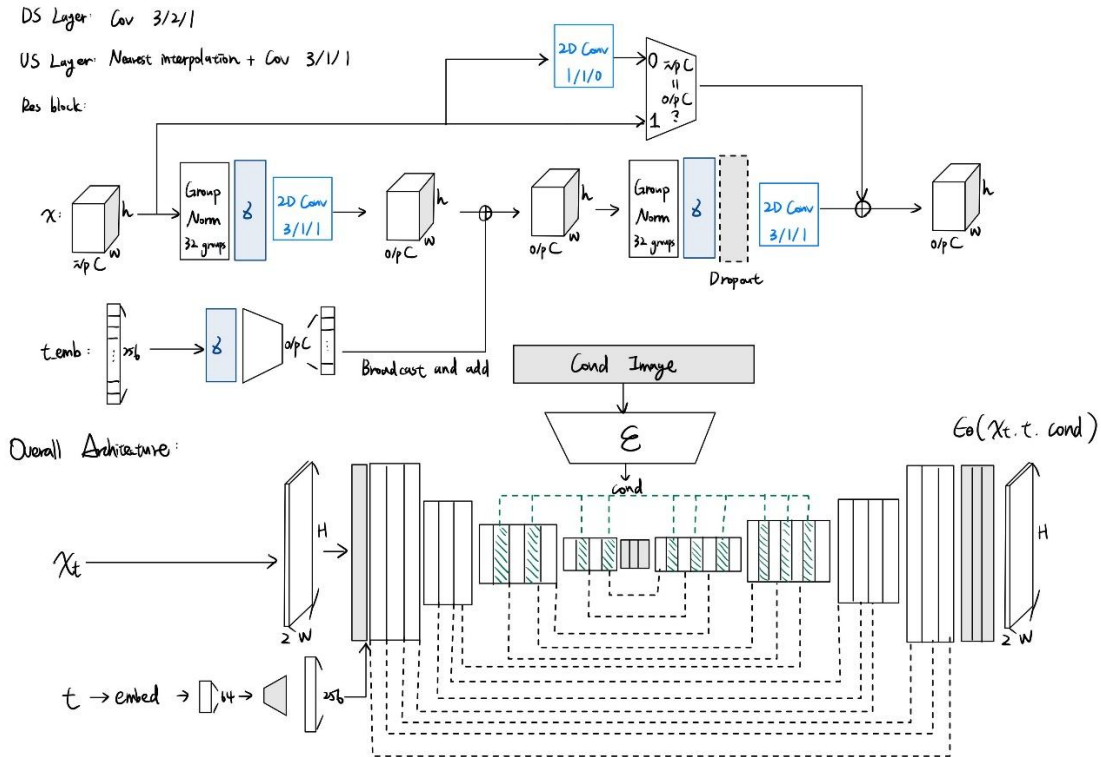
The following figure shows the final architecture and brief training/inference process:



The detailed structure of layers consisting of backbone UNet is:



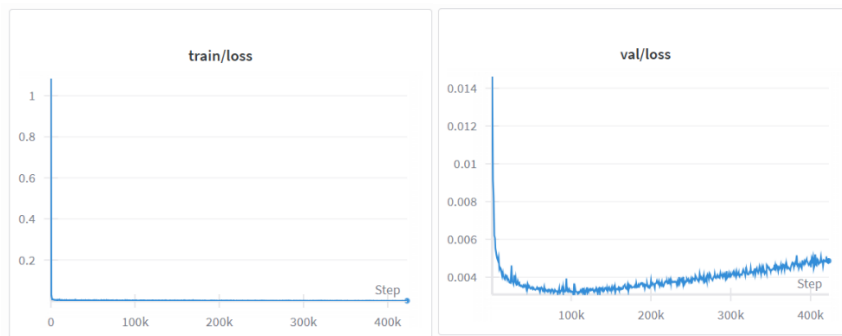
The design of residual block and the complete structure of denoising UNet:



During training, each batch has 16 items, while each item contains segment tensor with shape (2, 128, 128) and autoregressive condition tensor with shape (2, 204, 128). For a single song, each segment's start index is chosen every 64 steps (ticks). Therefore, different segments may have overlapping parts.

Loss function: $l = \|\epsilon - \epsilon_\theta(x_t, t, cond)\|_2^2$, ϵ : real noise, ϵ_θ : denoising UNet with condition encoder

Number of epochs = 500, the model will be trained on every batch in a single epoch. Below shows the loss curve on training set and validation set during training:



We can observe that overfitting occurs, so always the checkpoint with the best (the lowest) validation loss is used in following analysis.

- Generation / Inpainting tasks

These tasks are to ensure the diffusion model is successfully trained. There are three different tasks:

1. Song generation without autoregressive conditions:

Literally, generate a long song (512 ticks) from the beginning, each time the model will generate length-128 music segment. The second half, length 64 ticks, serves as the background for next generation. In other words, through local inpainting the whole song is produced.

2. Song generation with autoregressive conditions:

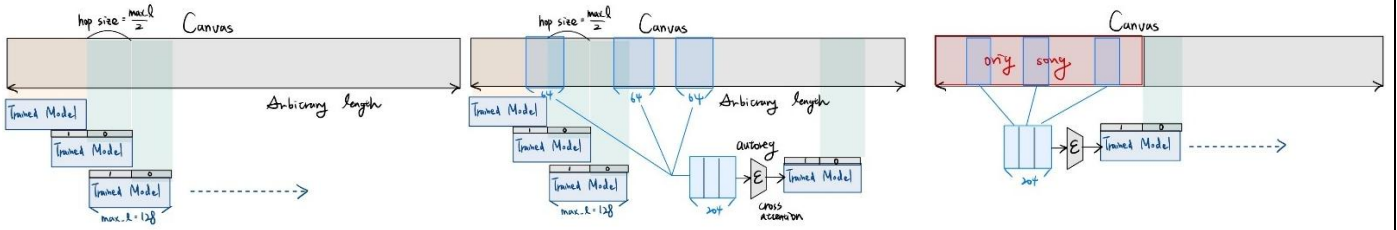
Like the first task, while autoregressive conditions are taken into consideration. The condition selection

rule is to choose 1~3 previous segments of length 64.

3. Song inpainting:

The first half (256 ticks) is filled with a real song, and the model generates the second half.

The detailed mechanisms of the three above-mentioned tasks are listed below:



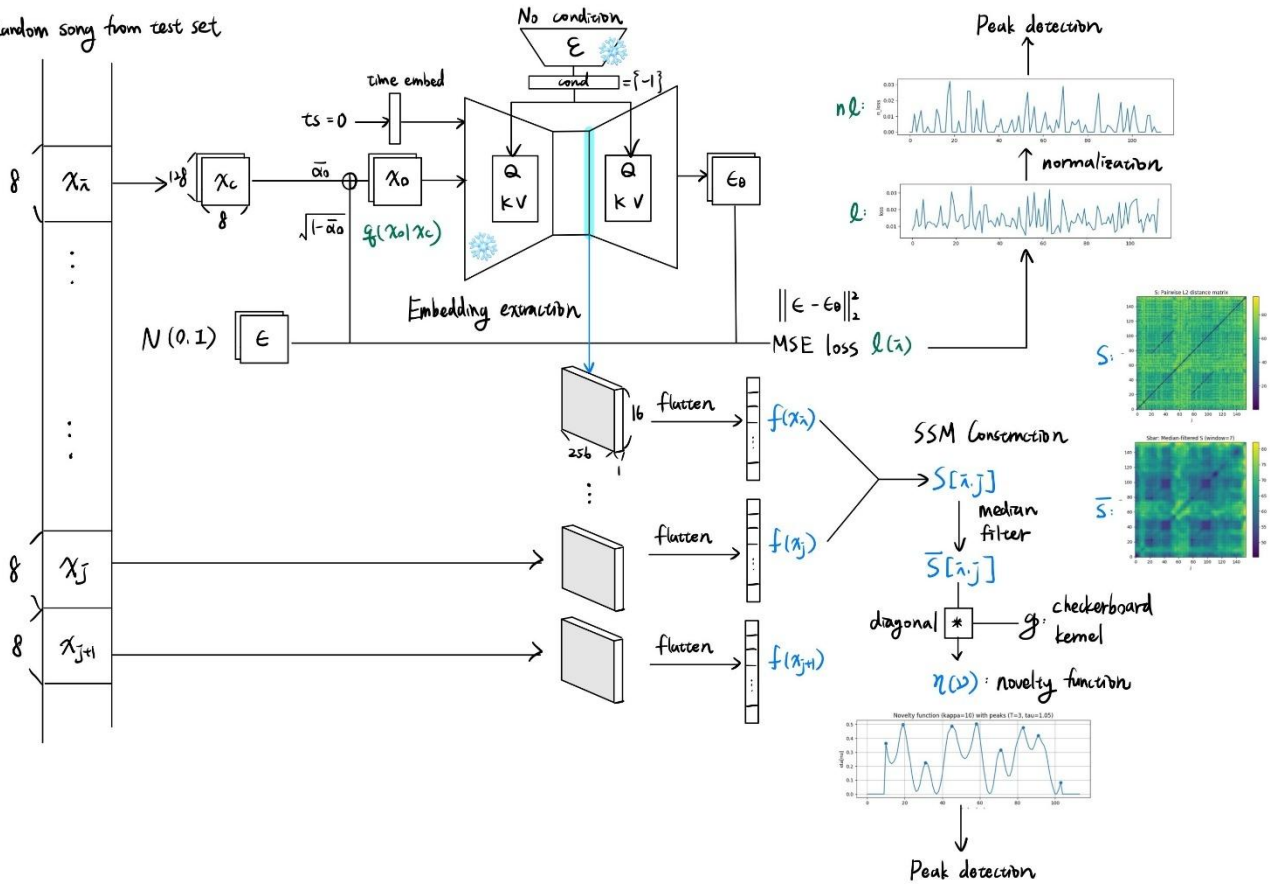
The quality of generation results is overall acceptable but lack some good song structure, and sometimes will have unharmony between chords in accompaniment and main melody. The interface (the start position of inpainting) won't have too obvious discontinuity. As the result, the diffusion model successfully learn how to generate music.

- Song segmentation task:

With reference to two papers mentioned in music segmentation part of paper survey, I propose two potential boundary detection methods with the trained diffusion model, the architecture:

Segmentation Task

Random song from test set



Both methods are unsupervised, details will be described in the following:

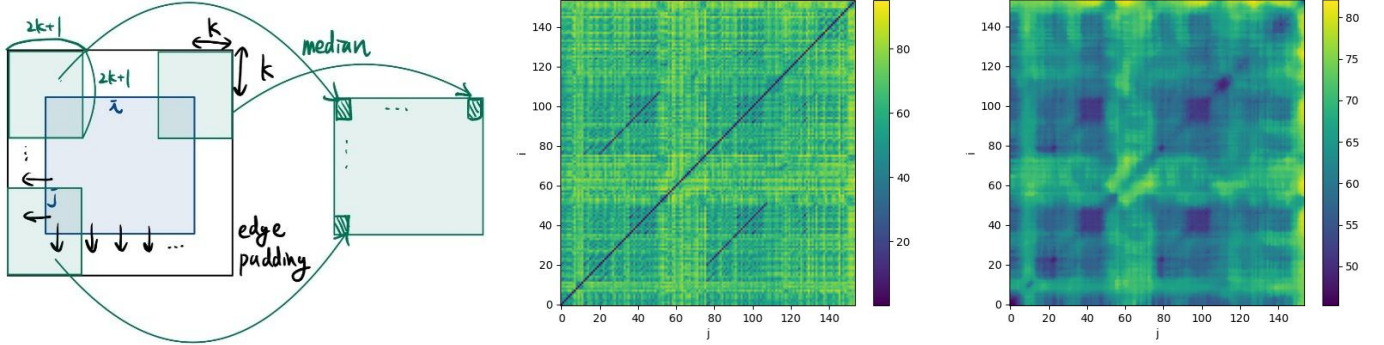
1. Embedding extraction with construction of SSM:

Our embedding is extracted from the output of middle block of UNet, while the input is a slightly noised song segment of length 8, time step equals 0, without any autoregressive condition. After being flattened,

the embedding vector is of size 4096. Then the 2-norm distance between different segment is measured, constructing matrix S , f denotes input + middle blocks:

$$S[i, j] = \|f(x_i^0, ts = 0) - f(x_j^0, ts = 0)\|_2, \bar{S} = S_{edge-padding} * f_M$$

Follow the convention in the paper, I further impose a median filter with kernel size = 7 ($k=3$) and edge padding of 3. The larger the filter is, the smoother the result will be but will loss more details. Below shows how median filter works and a random example $S/\bar{S}$:



Finally, I use a checkerboard kernel $\mathcal{G}$ to derive novelty function η , and do further process before local peak finding, the peak positions are exactly those boundary positions:

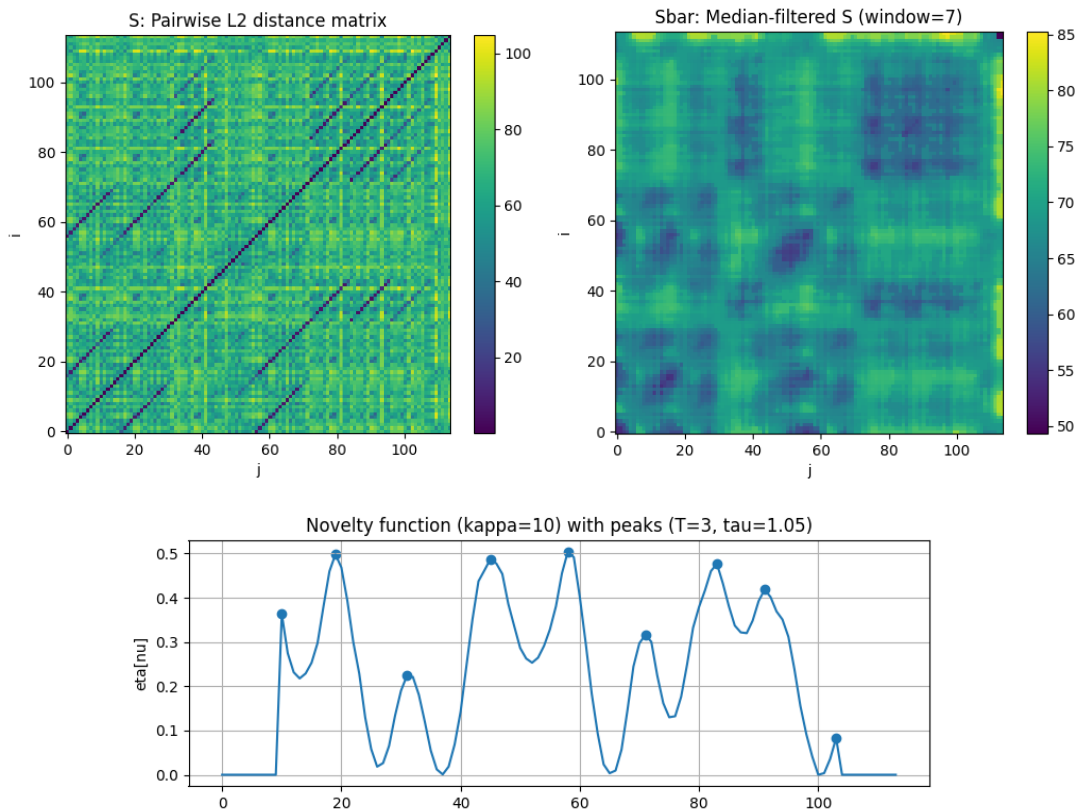
$$\mathcal{G}[i, j] = -sgn(i) \cdot sgn(j) \cdot \exp(-(i-j)^2/\sigma), \text{ where } -\kappa \leq i, j \leq \kappa$$

$$\eta(v) = \sum_{i=-\kappa}^{\kappa} \sum_{j=-\kappa}^{\kappa} \bar{S}[v+i, v+j] \cdot \mathcal{G}[i, j]; \bar{\eta}(v) = [\eta(v) - \min(\eta_{\neq 0})]/\alpha, \alpha : \text{scaling const}$$

Result analysis for song 0003 (4th song in pop909 dataset):

This song belongs to test set, which means that the model never sees it during training and validating.

Below shows the calculated SSM $S, \bar{S}$ and novelty function $\bar{\eta}$:

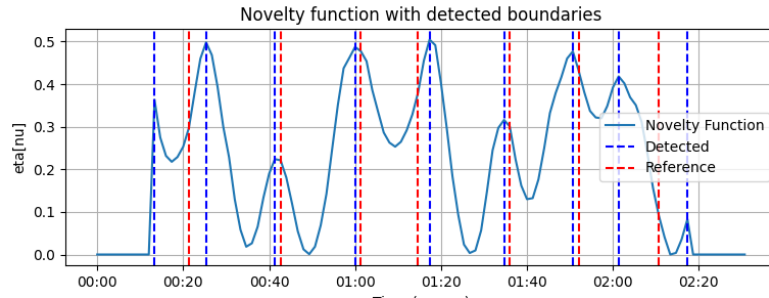


Thanks to the human labelled segmentation of all pop909 dataset's songs from the codebase of whole song hierarchical generation, I get to know the reference segment boundary's positions of 0003:

A8A8B7x5A8B6B7o7

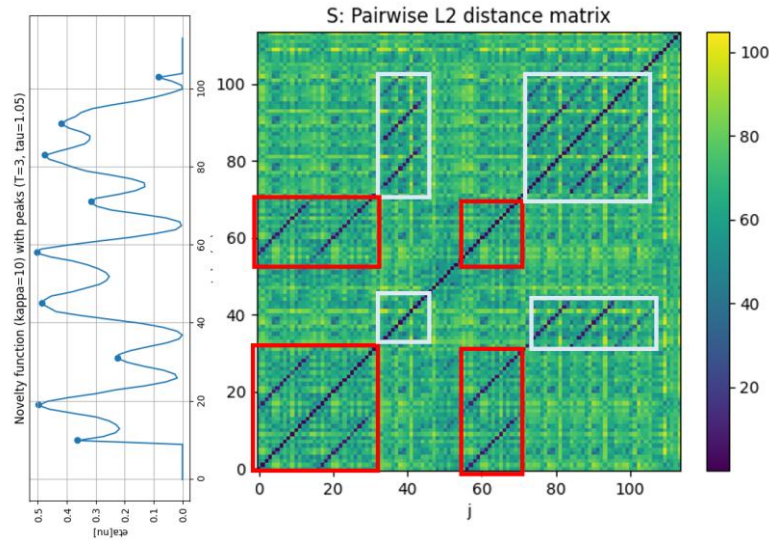
“A” represents verse section, “B” represents chorus section, “o” is for outro section, “x” is others.

Compare these reference peaks (red) with our predicted boundaries (blue):



Within a range of tolerance, we can simply calculate the accuracy and get the result TP = 6, FN = 1, FP = 3, precision = 0.67, recall = 0.86, and f1 = 0.75.

In addition, we can take a deeper look on SSM's pattern, especially not diagonal regions:



Ref: A8A8B7x5A8B6B7o7

As we can see, the red regions can correspond to “A” type segments: verse section, while light blue regions correspond to “B” type segments: chorus section. In addition to the **block structure** along the diagonal axis (blue shaded: lower value due to close distance, high similarity) which symbols different segments, the **path structure** reflects the high similarity between different segments. In other words, **the repetition of chorus or verse parts within a song**.

In conclusion, we can find the **potential of using embeddings from diffusion model to find segment boundaries**. Through observation from self-similarity matrix, it can show **clear song structure**. Moreover, when examining the time stamps of the predicted boundaries while listening to the song, I find that **most of these boundaries reasonably divide the song into distinct and meaningful segments**.

2. Temporal error prediction:

Instead of token classification error, to correspond to the nature of DDPM, here I interpret “error” as the “**denoising**” loss. In other words, for the prediction on next segment, firstly add some noise (default I choose time step=0) ϵ , and compare with the predicted noise $\epsilon_{\theta}(x_j^{ts}, ts)$ to calculate the loss:

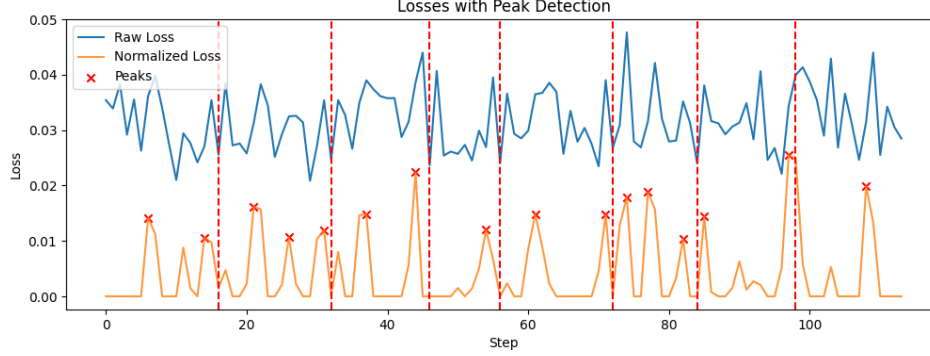
$$\mathcal{L}(j) = MSE = \frac{\|\epsilon - \epsilon_{\theta}(x_j^{ts}, ts)\|_2^2}{d}, d = CHW$$

And then use the normalization method to smooth the loss curve:

$$n\mathcal{L}(j) = \max(0, a \cdot \mathcal{L}(j+1) + \mathcal{L}(j) - \mathcal{L}(j-1) - b \cdot \mathcal{L}(j-2))$$

where $a=1.0$, $b=1.0$ are some hyper parameters.

Once we get the normalized loss $n\mathcal{L}$, we can do peak detection, similarly take song 0003 as the example:



From the above figure, the song is divided into **smaller segments** than those predicted by embedding extraction. Near the reference peaks (red dotted lines), normalized loss curve (orange) does appear high peaks. Another interesting discovery is that loss peaks occur more frequently on verse sections. I believe the **frequency of peak occurrence** will be important information on MIR task.

III. Conclusion and Future Works

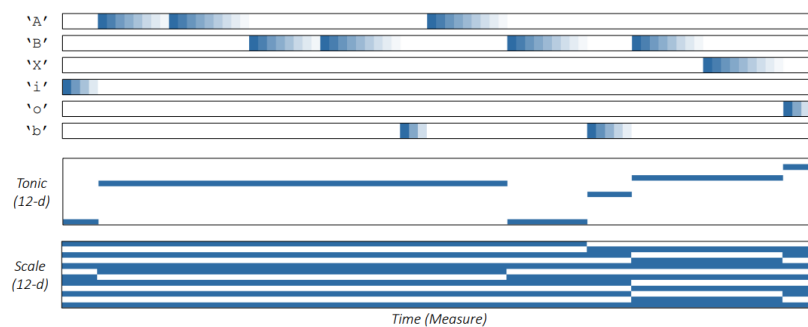
- I train a (lightweight) diffusion model to **generate/inpaint** music with **autoregressive conditioning**
- From the discussion above, the **embedding** of diffusion model's UNet does learn something about song segmentation
- Embedding **extraction with SSM** is seemingly a better method on music segmentation than **temporal error prediction**, but I think both have potentials
- However, to reach an ideal song segmentation performance, I think we should figure out a proper way to interpret the SSM, maybe a **better kernel that can concisely spot where the boundaries are**, or simply **not use 2 norm distance** as the measurement. Similarly, **how to interpret the denoising loss remains an open problem**
- **All parameters**, ranging from the size of median filter kernel to the rule for selecting local peaks, **should be considered more carefully**
- Furthermore, the **hidden information** inside SSM like the **extent of similarity between different segments** haven't been exploited yet
- Also, we should also compare **different models with different configurations on how do they affect the segmentation performance**, and also include **autoregressive conditioned embedding analyzations**
- Reconsider the pros and cons of **using both training sets: pop909 and pop1k7**. Since they have **different styles of music** which may affect the performance of diffusion model. On the other hand, we can try thinking how to **clean the midi of pop1k7 to reach the same extent of alignment as those in pop909**
- To improve generation quality, think twice when **dealing with the ending part of songs in**

training set

- It's important to **make a thorough and systematic evaluation** on music segmentation on more other songs with these different methods
- The data format shouldn't be limited to symbolic midi files but **mel spectrogram/waveform instead**, which will make the model more **flexible and applicable**. It's worth doing deeper/broader research
- I believe diffusion model can do a great job on **supervised task** if there are **sufficient and well-labelled training data**

IV. Interesting Topics

I've spent a whole week investigating theories behind Schrodinger bridge, since from another perspective, song segmentation is to learn the relationship between two distributions, one is music data, the other one is segment information. This involves supervised learning given that we are capable of transforming music into images (piano rolls as well as spectrograms) and converting segmentation information into images, which is inspired from the first level language in whole song hierarchical generation:



Schrodinger bridge will learn a bidirectional probability evolution between two distinct distributions, and is highly related to optimal transport problems which learns the joint probability distribution that minimizes the transport cost. Another reason why I think Schrodinger bridge might be a more effective method is that the raw music data serves as the direct input instead of conditions, which indicates that the model can take advantage of complete information of input song.

However, I later find out it is quite time consuming to fathom the mysteries and theories behind Schrodinger problems since it requires a wide range of mathematical backgrounds, like Ito calculus, random process, and so on.

Furthermore, when doing the recent paper survey: I2SB, I find that the two different distributions still fall on the same domain (like one is clean image, the other is blurred image), and in order to make the training feasible, there are always some math tricks or special constraints like specifically designed boundary conditions, i.e. the start and end marginal distributions.

Nevertheless, the Schrödinger Bridge remains highly interesting and deserves further investigation...

- Langevin Equation: $\frac{\partial p}{\partial t} = -\frac{\partial}{\partial x} (A(x, t)p(x, t)) + \frac{1}{2} \frac{\partial^2}{\partial x^2} (B(x, t)^2 p(x, t))$
- Fokker Planck Equation: $\frac{\partial p(x, t)}{\partial t} = -\frac{\partial}{\partial x} [a_t^{(1)}(x) \cdot p(x, t)] + \frac{1}{2} \frac{\partial^2}{\partial x^2} [a_t^{(2)}(x) \cdot p(x, t)]$